

Desenvolvimento de aplicações Web

Trabalho Prático Nº 2

Desenvolvimento Web – PERN Stack (Fullstack: Express + React + PostgreSQL)

1. Objetivo do Trabalho

O objetivo deste trabalho é desenvolver uma **aplicação fullstack completa** utilizando **Express (Node.js), React e PostgreSQL**, aplicando todos os conceitos teóricos e práticos abordados nas aulas.

Cada grupo terá **um dos tem** listados abaixo e desenvolverá uma solução funcional que inclua:

1. **Modelação da Base de Dados** (diagrama ER + SQL de criação).
2. **Backend Express** com estrutura de pastas organizada.
3. **Validação de dados** com middleware.
4. **Tratamento de erros** centralizado.
5. **Autenticação básica**.
6. **PostgreSQL** para persistência de dados (rotinas CRUD: Create, Read, Update, Delete).
7. **Frontend React** com componentes, useState, useEffect, fetch, e integração com a API.

2. Temas

Tema 1 – Sistema de Gestão de Tarefas de Equipa: **Aurien, Herisene, Elvin**

Objetivo: Sistema para gerir tarefas de uma equipa, com atribuição a usuários, status e prazos.

Tema 2 – Sistema de Gestão de Reservas de Salas: **Kelly, Jarni, Leonardo**

Objetivo: Sistema para reservar salas de reunião, com usuário, sala e data/hora.

Tema 3 – Plataforma de Gestão de Pedidos de Compras: **Bruno, João, Aulindo**

Objetivo: Sistema para gerir pedidos de compras de produtos.

Tema 4 – Sistema de Gestão de Agendamentos de Consultas: **Eder, Marcos**

Objetivo: Sistema para agendar consultas com profissionais de saúde.

3. Requisitos Técnicos do Backend (Express + PostgreSQL)

3.1. Modelação da Base de Dados (Obrigatório)

- **Entregar um diagrama ER** (Entity-Relationship) da base de dados (pode ser feito em ferramentas como Draw.io, dbdiagram.io, mysql workbench).
- **Entregar o script SQL completo** para criar a base de dados, tabelas, chaves primárias e estrangeiras.
- Inserir pelo menos **5 registros iniciais** em cada tabela.

3.3. Rotas e Controllers

A aplicação deve implementar **rotas CRUD completas** para o recurso principal:

- GET /api/<recurso> → listar todos os registros.
- GET /api/<recurso>/:id → listar um registro específico.
- POST /api/<recurso> → criar novo registro.
- PUT /api/<recurso>/:id → atualizar um registro.
- DELETE /api/<recurso>/:id → apagar um registro.

Todos os handlers das rotas devem ser **funções isoladas em controllers** (nunca diretamente no server.js).

3.4. Middleware de Validação

- Deve ser criado um middleware de validação em src/middleware/validation.middleware.js.
- O middleware deve:
 - Verificar **campos obrigatórios** no corpo da requisição
 - Verificar **regras de negócio**
 - Retornar status 400 com mensagem clara em caso de dados inválidos.

3.5. Middleware de Tratamento de Erros Centralizado

- Deve ser criado um middleware de erro no `server.js`, **após todas as rotas**.
- O middleware deve ter a assinatura:

`(err, req, res, next) => { ... }`

- O middleware deve:
 - Tratar erros lançados pelos controllers.
 - Definir status (ex: `err.status` ou 500).
 - Enviar resposta padronizada: `res.status(status).json({ mensagem: ... })`.
 - Registrar o erro no `console.error()`.

3.6. Middleware de Autenticação

- Deve ser criado um middleware de autenticação em `src/middleware/auth.middleware.js`.
- O middleware deve:
 - Ler o header `Authorization`.
 - Verificar se é do tipo `Bearer <token>`.
 - Validar o **token**.

3.7. Integração com PostgreSQL

- Deve ser criado um banco de dados PostgreSQL.
- Os controllers devem usar **models** (`src/models/<recurso>.model.js`) para acessar o banco.
- Consulta SQL básica: `SELECT`, `INSERT`, `UPDATE`, `DELETE`.

4. Requisitos Técnicos do Frontend (React)

4.1. Estrutura do Projeto React

- O projeto React deve ser organizado com as seguintes pastas:
 - src/components/ → componentes reutilizáveis.
 - src/pages/ → páginas principais.

4.2. Componentes React

- Componente devem recebe os dados via props e os exibir.
- Deve usar:
 - useState para gerenciar estados.
 - useEffect para qualquer efeito colateral.
 - fetch para chamar a API.
 - try/catch para tratar erros da API.

4.3. Integração com a API

- O frontend deve:
 - Chamada GET /api/<recurso> para listar todos os registros.
 - Chamada POST /api/<recurso> para criar novo registro.
 - Chamada PUT /api/<recurso>/:id para atualizar registro.
 - Chamada DELETE /api/<recurso>/:id para apagar registro.
- O frontend deve exibir mensagens de erro ou sucesso ao usuário.

5. Resultado Esperado

No final do trabalho, o grupo deverá entregar:

1. **Repositório GitHub** com o código do projeto (backend + frontend) e enviar o código compactado em ficheiro zip via email (**willianpires.dev@gmail.com**).
2. **README.md** com:
 - Instruções de instalação e execução.
 - Endereço base da.
 - Exemplos de requisições com Postman ou cURL.
 - Token de autenticação para testar as rotas privadas.
 - **Diagrama ER** da base de dados (imagem ou link).
 - **Script SQL** completo para criar a base de dados.
3. **Base de dados PostgreSQL** com pelo menos **5 registros iniciais** em cada tabela.
4. **Aplicação funcional** com:
 - Listagem de todos os registros.
 - Criação de novo registro.
 - Edição de registro existente.
 - Exclusão de registro.
 - Autenticação nas rotas.
 - Tratamento de erros no frontend e backend.

6. Critérios de Avaliação

Critério	Peso
Modelação da Base de Dados (diagrama ER + script SQL)	15%
Estrutura do projeto Express (controllers, routes, middleware, models)	20%
Rotas CRUD completas (GET, POST, PUT, DELETE)	15%
Middleware de validação (campos obrigatórios, regras de negócio)	10%
Middleware de tratamento de erros centralizado (err, req, res, next)	5%
Middleware de autenticação (Bearer token, status 401)	5%
Integração com PostgreSQL (model, consultas SQL)	10%
Frontend React (componentes, useState, useEffect, fetch)	10%
Integração frontend com backend (API, autenticação)	5%
Documentação (README.md, instruções de uso)	5%

7. Prazo de Entrega

- **Data limite:** 15/06/2026
- **Apresentação:** cada grupo terá 15 minutos para apresentar a aplicação e explicar:
 - Estrutura do projeto.
 - Modelação da base de dados.
 - Implementação dos middlewares.
 - Integração com PostgreSQL.
 - Integração frontend com backend.